```
    std::unique_ptr<node> wait_pop_head()
    {
        std::unique_lock<std::mutex> head_lock(wait_for_data());     <-- 4
        return pop_head();
    }

    std::unique_ptr<node> wait_pop_head(T& value)
    {
        std::unique_lock<std::mutex> head_lock(wait_for_data());     <-- 5
        value=std::move(*head->data);
        return pop_head();
    }
public:
    std::shared_ptr<T> wait_and_pop()
    {
        std::unique_ptr<node> const old_head=wait_pop_head();
        return old_head->data;
    }

    void wait_and_pop(T& value)
    {
        std::unique_ptr<node> const old_head=wait_pop_head(value);
    }
};
```

The implementation of the pop side shown in listing 6.9 has several little helper functions to simplify the code and reduce duplication, such as `pop_head()` ❶ and `wait_for_data()` ❷, which modify the list to remove the head item and wait for the queue to have some data to pop, respectively. `wait_for_data()` is particularly noteworthy, because not only does it wait on the condition variable using a lambda function for the predicate, but it also returns the lock instance to the caller ❸. This is to ensure that the same lock is held while the data is modified by the relevant `wait_pop_head()` overload ❹, ❺. `pop_head()` is also reused by the `try_pop()` code shown in the next listing.

Listing 6.10. A thread-safe queue with locking and waiting: `try_pop()` and `empty()`

```
template<typename T>
class threadsafe_queue
{
private:
    std::unique_ptr<node> try_pop_head()
    {
        std::lock_guard<std::mutex> head_lock(head_mutex);
        if(head.get()==get_tail())
```